TCSS 342 – Data Structures

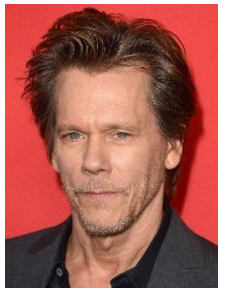
Graphs: Definition and Implementation

Social Networks

- Acquaintance/friendship, influence, collaboration



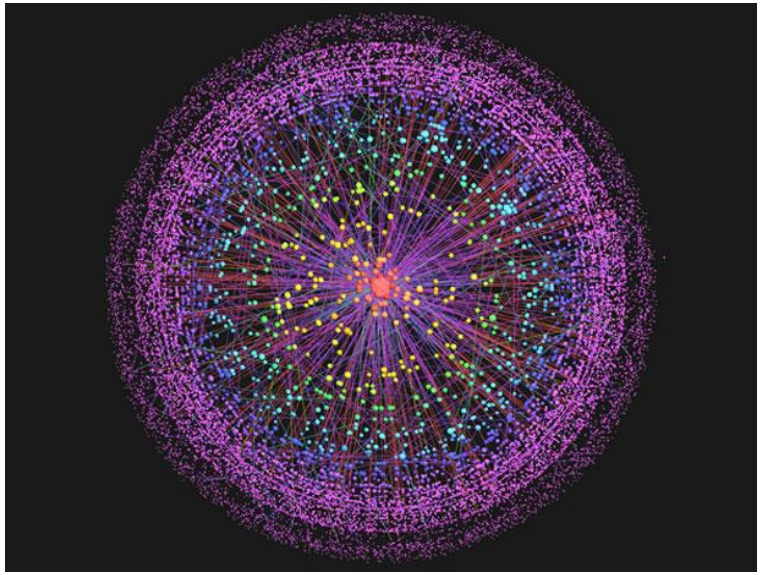




Computer Networks

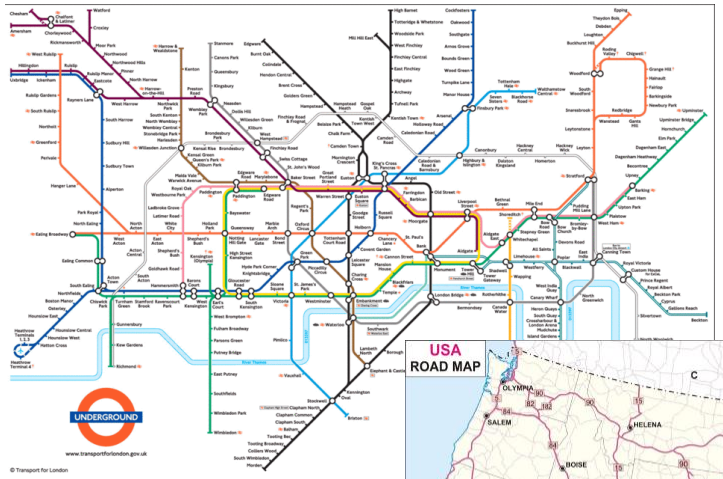
- Home networks, WWW, P2P, gaming networks





Transportation Networks

- Public transit, road maps, flight maps.



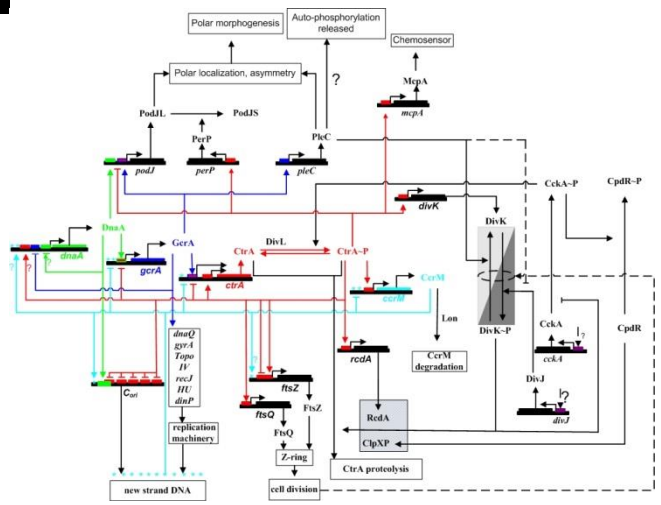


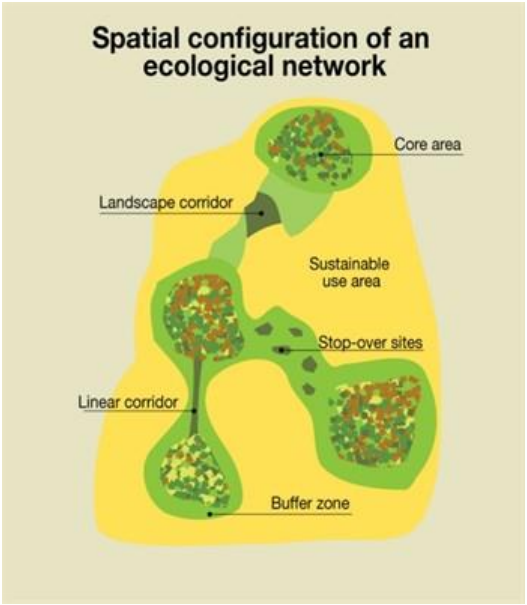


Biological Networks

- Neural networks, genetic networks, ecological networks.

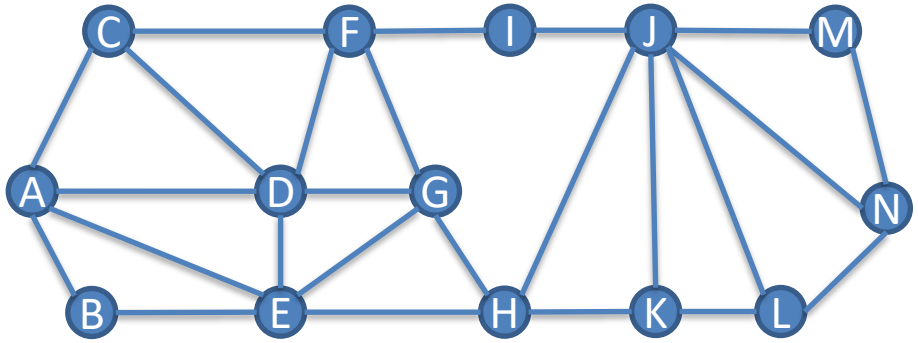






Undirected Graph

- A graph is an ordered pair, $G = (V, E)$, comprised of a set of vertices V and a set of edges E such that:
  - $V = \{v_1, v_2, \ldots, v_n\}$ is the set of $n \geq 0$ vertices.
  - $E \subseteq \{\{u, v\} | u, v \in V\}$ is the set of edges.
- This is the simplest variety of graph and is also called an undirected simple graph.
  - An undirected graph associates no direction with edges.
- A vertex is also known as a node, point, or state.
  - Not every vertex in the graph must belong to an edge.
- An edge is also known as a connection, link, line, branch or transition.
  - In an edge $\{u, v\}$ the vertices u and v are called endpoints.
  - The vertices u and v are said to be joined by the edge.
  - The vertices u and v are said to be adjacent to and incident on one another.
  - An edge $\{u, u\}$ connecting a vertex to itself is called a loop or a self-edge and is not always permitted.



$$V = \{A, B, C, D, E, F, G, H, I, J, K, L, M, N\}$$

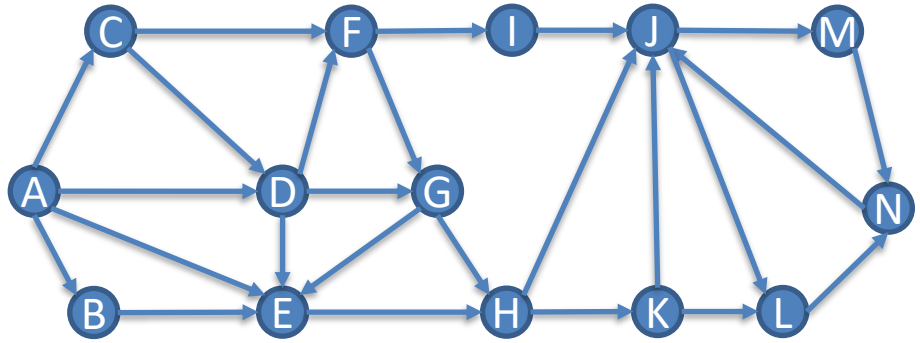
$$E = \left\{ \begin{matrix} \{A, B\}, \{A, C\}, \{A, D\}, \{A, E\}, \{B, E\}, \\ \{C, D\}, \{C, F\}, \{D, E\}, \{D, F\}, \{D, G\}, \\ \{E, G\}, \{E, H\}, \{F, G\}, \{F, I\}, \{G, H\}, \\ \{H, J\}, \{H, K\}, \{I, J\}, \{J, K\}, \{J, L\}, \\ \{J, M\}, \{J, N\}, \{K, L\}, \{L, N\}, \{M, N\} \end{matrix} \right\}$$

Directed Graph

- A directed graph is an ordered pair, $G = (V, E)$, comprised of a set of vertices V and a set of edges E such that:
  - $V = \{v_1, v_2, \dots, v_n\}$ is the set of $n \geq 0$ vertices.
  - $E \subseteq \{(u, v) | u, v \in V\}$ is the set of edges.

- The only difference between an undirected graph and a directed graph is the direction associated with the edges.
  - Treating the edge as an ordered pair (u, v) instead of a set $\{u, v\}$ captures this difference.
  - In an undirected graph $\{u, v\} = \{v, u\}$.
  - In a directed graph $(u, v) \neq (v, u)$.

- The size of the vertex set, $n = |V|$, is called the order of the graph.
- The size of the edge set, $m = |E|$, is called the size of the graph.



$$V = \{A, B, C, D, E, F, G, H, I, J, K, L, M, N\}$$

$$E = \left\{ \begin{matrix} (A,B), (A,C), (A,D), (A,E), (B,E), \\ (C,D), (C,F), (D,E), (D,F), (D,G), \\ (G,E), (E,H), (F,G), (F,I), (G,H), \\ (H,J), (H,K), (I,J), (K,J), (J,L), \\ (J,M), (N,J), (K,L), (L,N), (M,N) \end{matrix} \right\}$$

Size of a Graph

- The size of the vertex set, $n = |V|$, is called the order of the graph.
- The size of the edge set, $m = |E|$, is called the size of the graph.

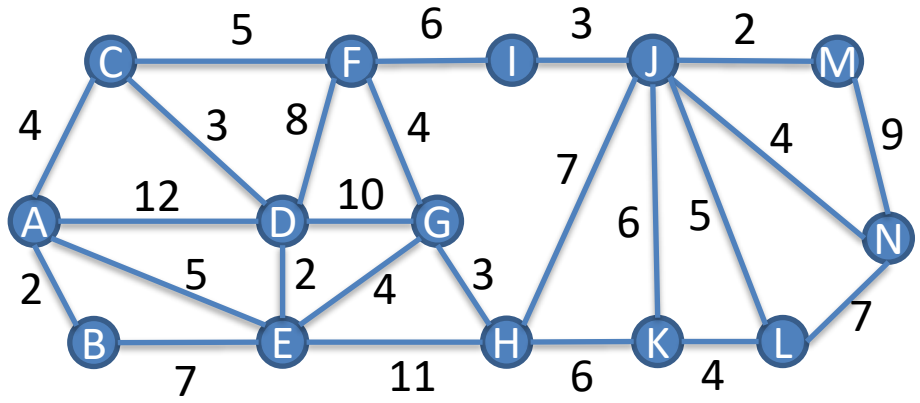
- The minimum size of a graph is 0 (given no other criteria).

- The maximum size of a graph depends on the kind of graph.
  - Undirected with no loops – $\frac{n(n-1)}{2}$
  - Undirected with loops – $\frac{n(n+1)}{2}$
  - Directed with no loops – $n^2 - n$
  - Directed with no loops – n^2

- A sparse graph is a graph with number of edges close to the minimum number of edges.
- A dense graph is a graph with number of edges close to the maximum number of edges.
  - The distinction between sparse and dense graphs is vague.

Weighted Graph

- A labelled (undirected or directed) graph is an ordered triple, $G = (V, E, l)$, comprised of a set of vertices V, a set of edges E, and a label function l such that:
  - $l: E \to L$ is a function that maps from the edge set to a set of labels.
- A weighted (undirected or directed) graph is an ordered triple, $G = (V, E, w)$, comprised of a set of vertices V, a set of edges E, and a weight function w such that:
  - $w: E \to \mathbb{R}$ is a function that maps from the edge set to a real number.
- A weighted graph is a labelled graph where the labels are numbers.



$$V = \{A, B, C, D, E, F, G, H, I, J, K, L, M, N\}$$

$$E = \left\{ \begin{matrix} (A,B), (A,C), (A,D), (A,E), (B,E), \\ (C,D), (C,F), (D,E), (D,F), (D,G), \\ (G,E), (E,H), (F,G), (F,I), (G,H), \\ (H,J), (H,K), (I,J), (K,J), (J,L), \\ (J,M), (N,J), (K,L), (L,N), (M,N) \end{matrix} \right\}$$

Graph Implementation

- As with any data structure we have a choice how to implement the graph.
  - We can implement the graph as a linked structure.
  - We can implement the graph using an underlying array.

- Adjacency lists – the graph is implemented as a linked structure.
  - We maintain a list of nodes.
  - Each node has a list of adjacent nodes.
  - In some cases each node has a list of edges that themselves contain two endpoints.
    - This is used when the edges need to carry data like a label or edge weight.
  - Works well for sparse graphs.

- Adjacency matrix – the edges of the graph are encoded in the entries in a $n \times n$ matrix.
  - We maintain a list of nodes.
  - Every entry of the $n \times n$ matrix corresponds to one of the n^2 potential edges.
  - Entries of the matrix can be Boolean, numerical (for weights) or of arbitrary types (for labels).
  - Undirected graphs are symmetric.
  - The diagonal of the matrix is reserved for self-edges (loops).
  - Works well for dense graphs.

TCSS 342 – Data Structures

Graphs: Definition and Representation

Graph Terminology

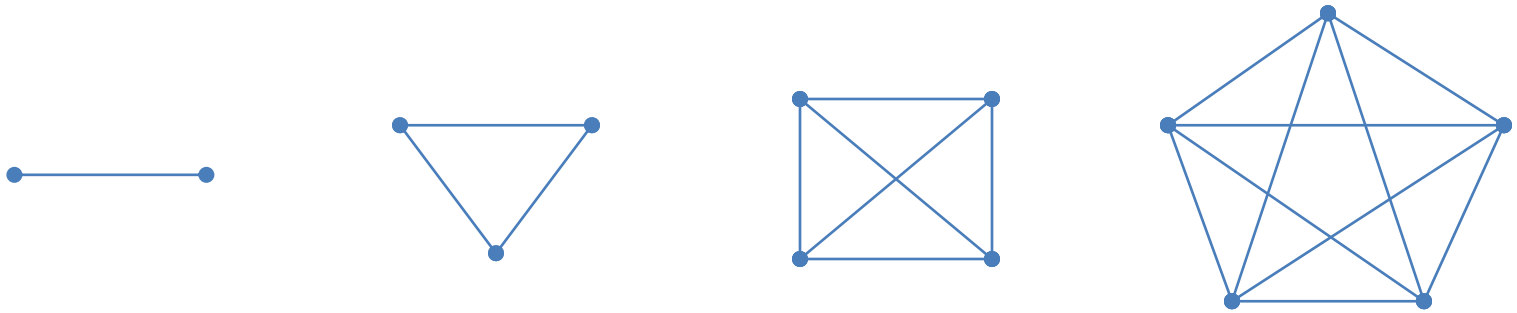
- Two vertices that are connected by an edge are said to be adjacent or incident on one another and are called neighbors.
  - The set of vertices incident on a vertex is called the vertex's neighborhood.
  - In directed graphs we use phrase like "adjacent to" and "adjacent from" to indicate the direction of the connection.
- The degree of a vertex is equal to the number of edges connected to the vertex.
  - In directed graphs we can distinguish the in-degree (number of connections coming into the vertex) from the out-degree (number of connections going out of the vertex).
- Handshaking Theorem: The sum of degrees of all vertices is even.
  - Proof: Every edge is counted twice, once for each end point. □

Paths and Cycles

- A path from u to v in a graph $G = (V, E)$ is a sequence of n vertices
$$(x_0 = u, x_1, x_2, \dots, x_{n-1} = v)$$
where $(x_i, x_{i+1}) \in E$ for all i from 0 to $n - 2$.
  - A cycle is a path where additionally (x_{n-1}, x_0).
- A simple path/cycle is a path/cycle where every vertex is unique (i.e. none repeated).
- A graph is called connected if there exists a path from every vertex to every other.
  - A directed graph is called weakly connected if the underlying undirected graph is connected.
  - A directed graph is called strongly connected if there exists a directed path from every vertex to every other vertex.

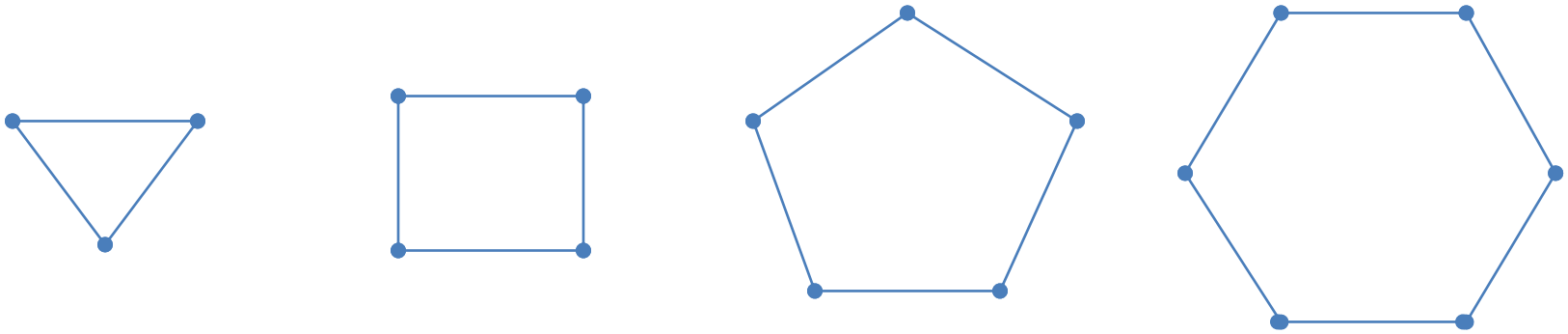
Complete Graphs

- A graph is called complete if every pair of vertices has an edge between them.
  - This is also called a clique.
  - The Clique Problem is the problem of finding the largest clique in a graph.



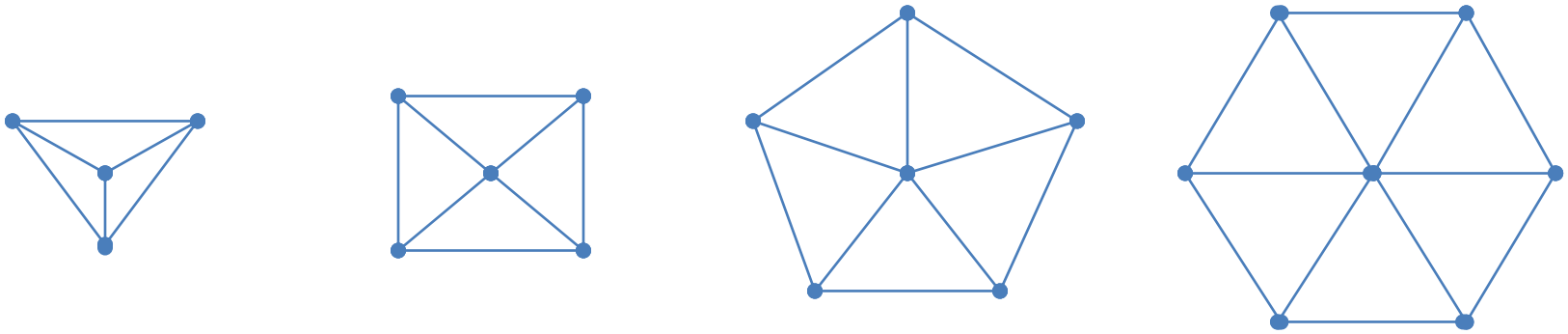
Cycles

- A graph is called a cycle or circuit if every vertex is connected to exactly two other vertices and there exists a path from any vertex to another.
  - A common problem is to find cycles is a graph.
    - Any cycle, longest cycle, shortest cycle, etc.



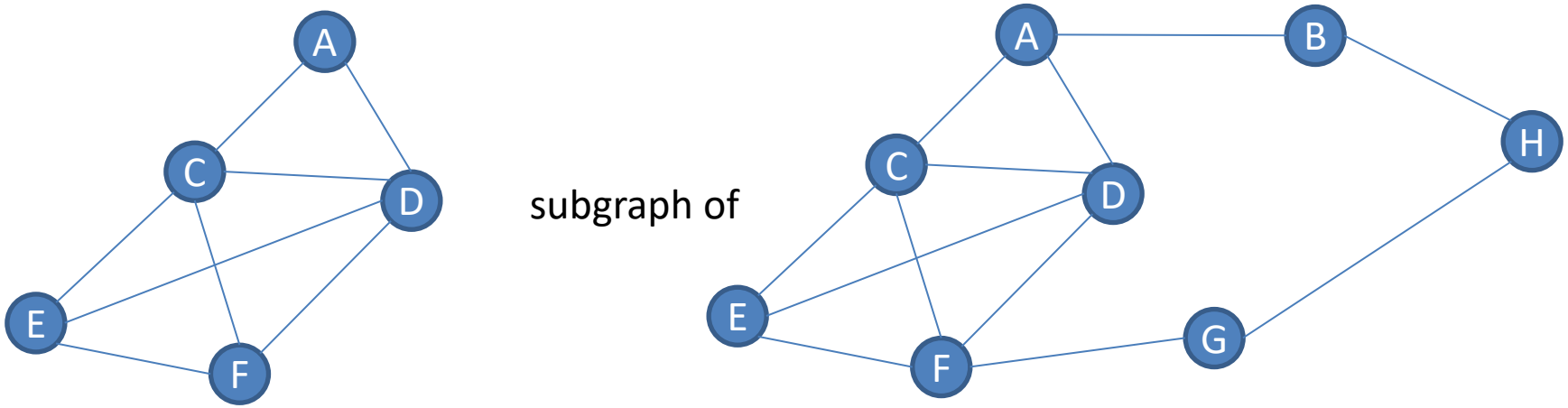
Wheel

- A graph is called a wheel if every non-central vertex is connected to exactly two other non-central vertices and to a single central vertex.



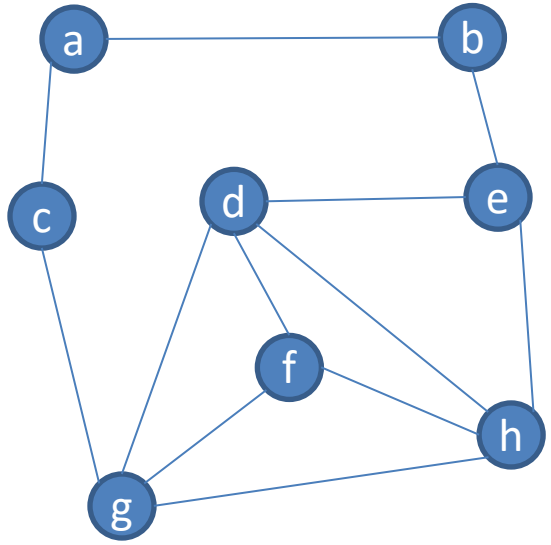
Subgraphs

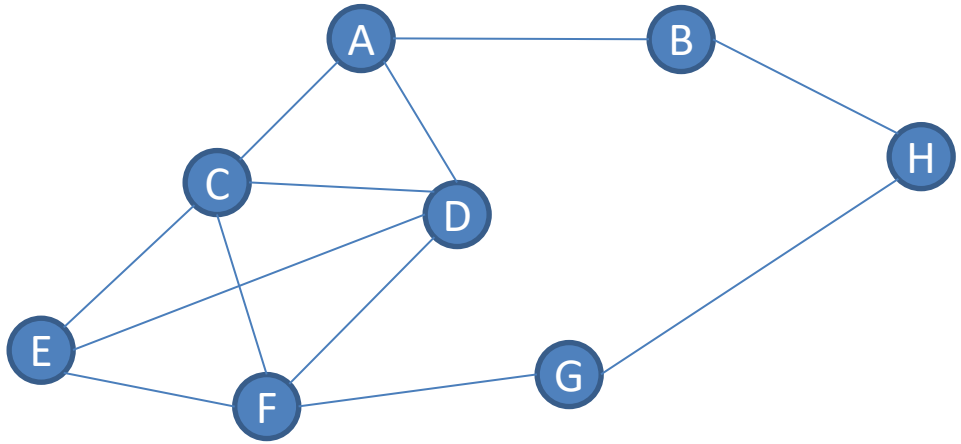
- If $G = (V, E)$ is a graph then $G' = (V', E')$ is a subgraph of G if $V' \subseteq V$ and $E' \subseteq E$ and for every $(u, v) \in E'$ we have that $u, v \in V$.
  - We can induce a subgraph by selecting only a subset of vertices V' and including every edge from the original edge set that has both endpoints in V'.



Graph Isomorphism

- Two graphs with the same shape are called isomorphic.
- Two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are isomorphic if a one-to-one mapping $m_V : V_1 \rightarrow V_2$ exists between the vertex sets and a one-to-one mapping $m_E : E_1 \rightarrow E_2$ exists between the edge sets.





Checking for Isomorphism

- Best known algorithm is exponential in the number of vertices in the graphs to be checked.
  - Easy checks can be done to rule out cases.
    - The vertex sets must be of the same size.
    - The edge sets must be of the same size.
    - The set of vertex degrees must be the same.
  - Any property that must remain the same across an isomorphism can be used to detect non-isomorphism and is called a graph invariant.
- To show isomorphism: find the one-to-one mapping that works.
- To show non-isomorphism: find an invariant that is violated.

Sub-graph Isomorphism

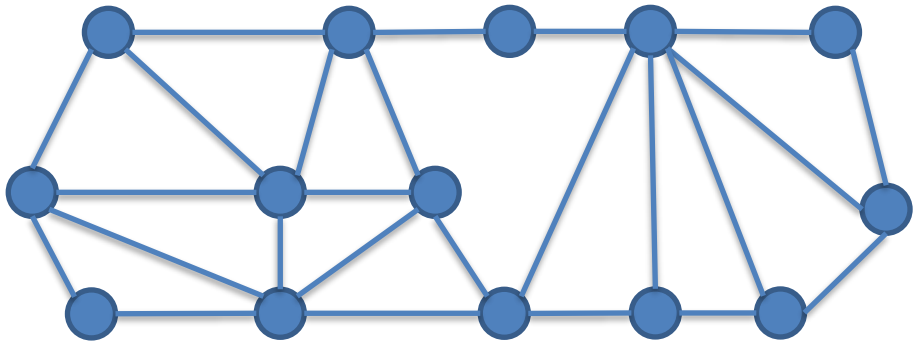
- Graph isomorphism is a special case of sub-graph isomorphism.
- Sub-Graph Isomorphism:
  - Input: $G_1 = (V_1, E_1), G_2 = (V_2, E_2)$ two graphs.
  - Outputs: A one-to-one map $m: V_1 \rightarrow V_2'$ where $V_2' \subseteq V_2$ and G_1 is isomorphic via m to the sub-graph induced by V_2' on G_2.
- Other special cases of this problem are:
  - Clique – find a sub-graph isomorphic to a complete graph.
  - Cycle – find a sub-graph isomorphic to a cycle.

TCSS 342 – Data Structures

Graph Terminology

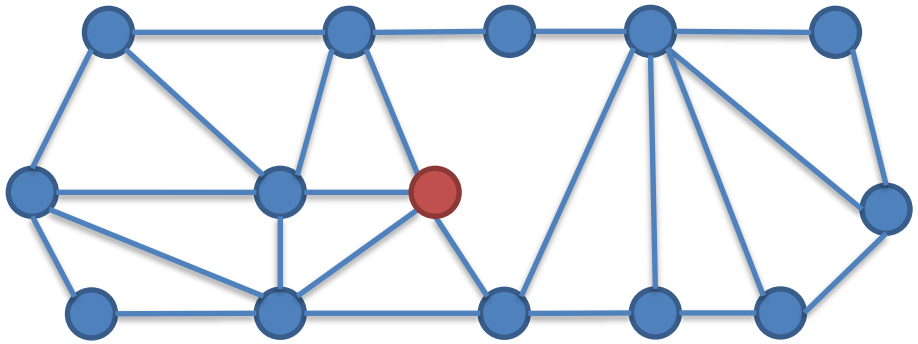
Graph Terminology

- A simple graph is an undirected graph with no loops.



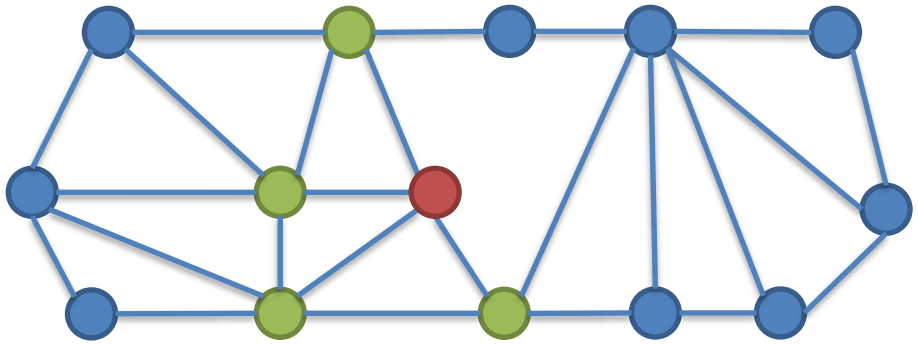
Graph Terminology

- The degree of a vertex is the number of edges incident on the vertex.
  - Alternatively, and equivalently, the degree of a vertex is the number of vertices adjacent to the vertex.
  - The degree of this vertex is 4.



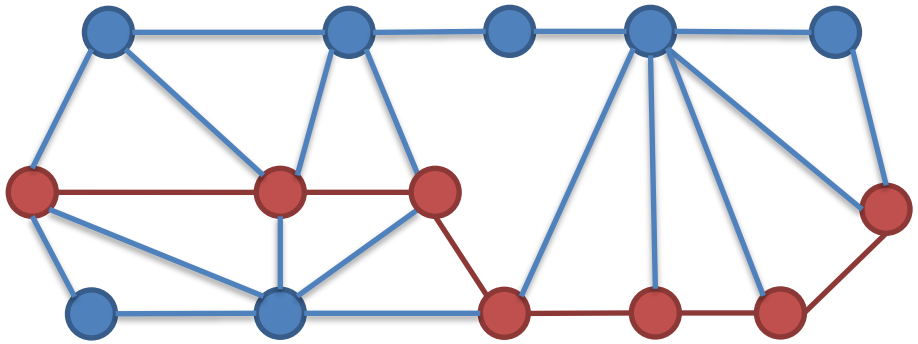
Graph Terminology

- The neighborhood of a vertex is the set of adjacent vertices.
  - The neighborhood of the red vertex is indicated in green.



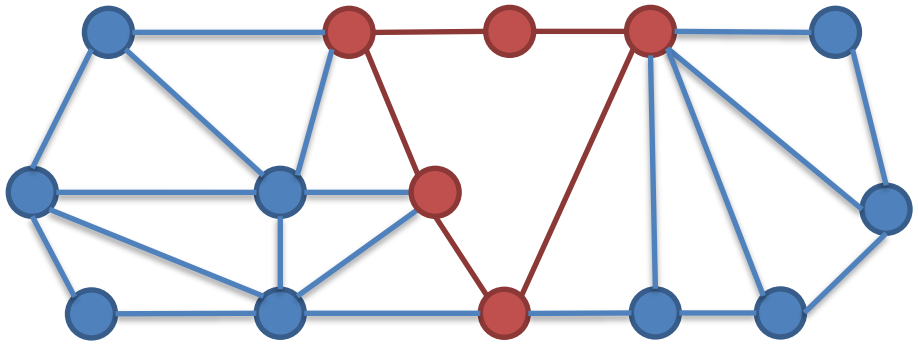
Graph Terminology

- A path is a sequence of vertices $(v_1, v_2, \ldots, v_n)$ such that
$$\{\{v_i, v_{i+1}\} | 1 \leq i < n\} \subseteq E.$$
  - A simple path does not repeat vertices.



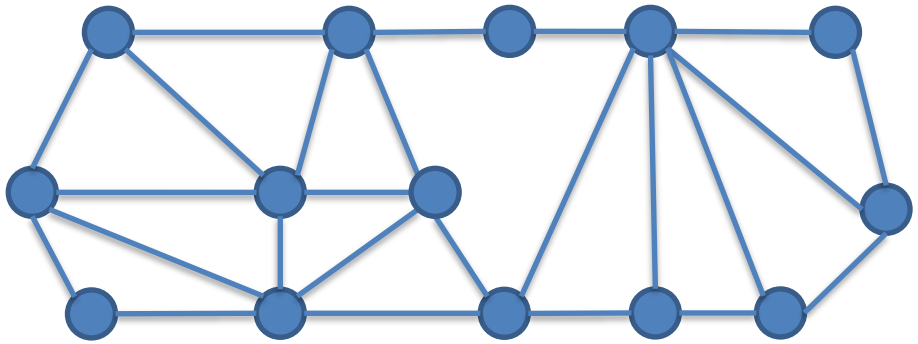
Graph Terminology

- A cycle is a sequence of vertices $(v_1, v_2, \ldots, v_n)$ such that
$$\{\{v_i, v_{i+1}\} | 1 \leq i < n\} \subseteq E \ \wedge \ \{v_n, v_1\} \in E.$$
  - A simple cycle does not repeat vertices.



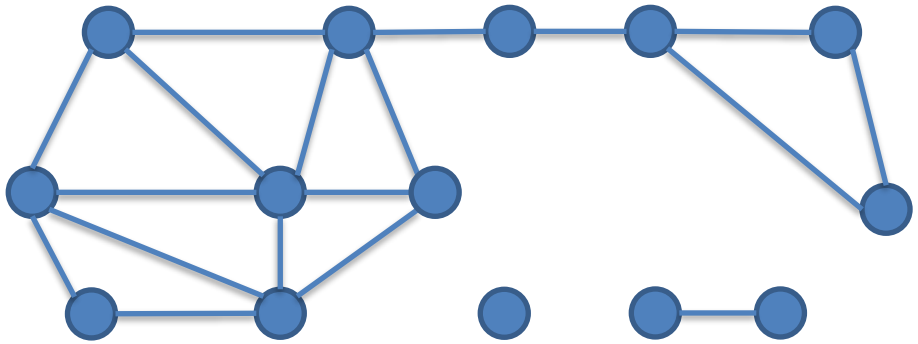
Graph Terminology

- A graph is connected if there exists a path from every vertex to every other vertex.
  - This graph is connected.



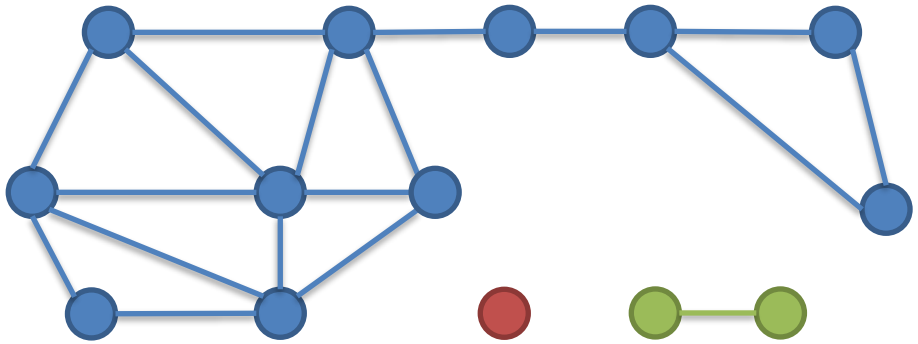
Graph Terminology

- A graph is connected if there exists a path from every vertex to every other vertex.
  - This graph is disconnected.



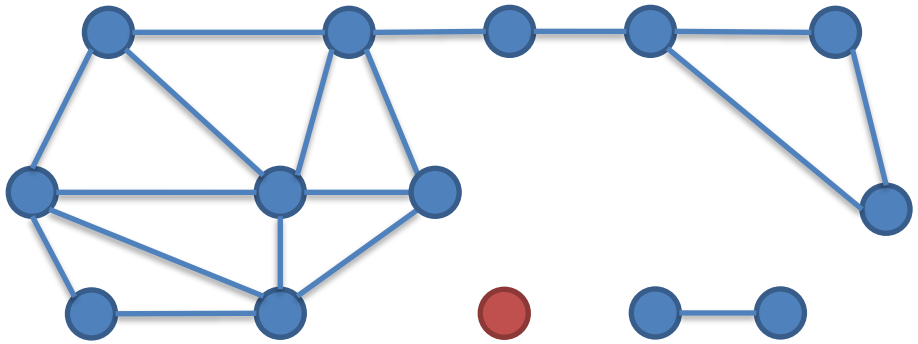
Graph Terminology

- We say a disconnected graph consists of connected components.
  - This graph consists of three connected components.



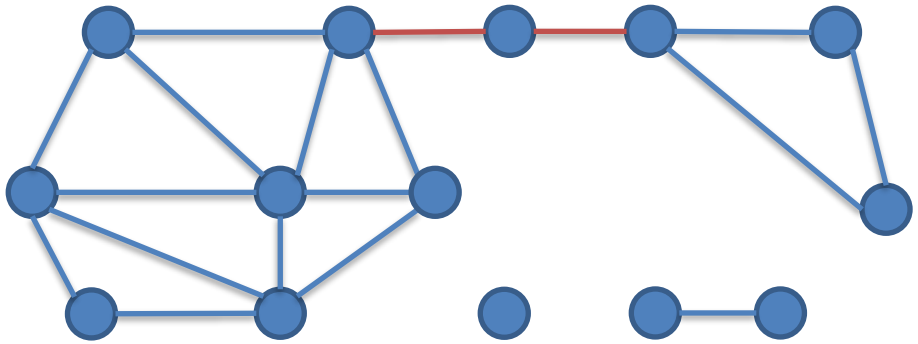
Graph Terminology

- We call a vertex isolated (or an island) if it is connected to no other vertices.
  - This vertex is an island.



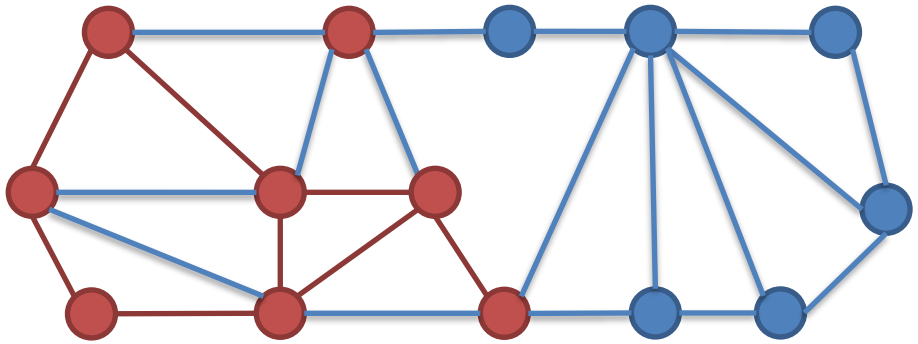
Graph Terminology

- We call an edge a bridge if removing it would disconnect a component.
  - Alternatively, a bridge is an edge that is not part of any cycle.
  - These edges are bridges.



Graph Terminology

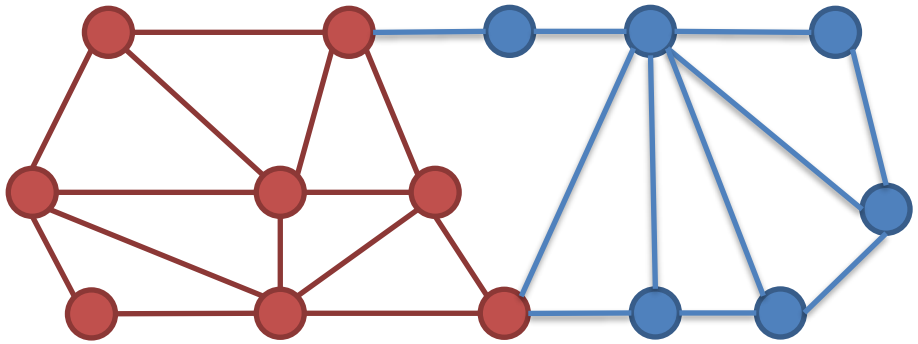
- A graph $G' = (V', E')$ is called a sub-graph of another graph $G = (V, E)$ if:
  - $V' \subseteq V$ and $E' \subseteq E$,
  - and, if $(u, v) \in E'$ then $u, v \in V'$.
- The red graph is a sub-graph of the entire graph.



Graph Terminology

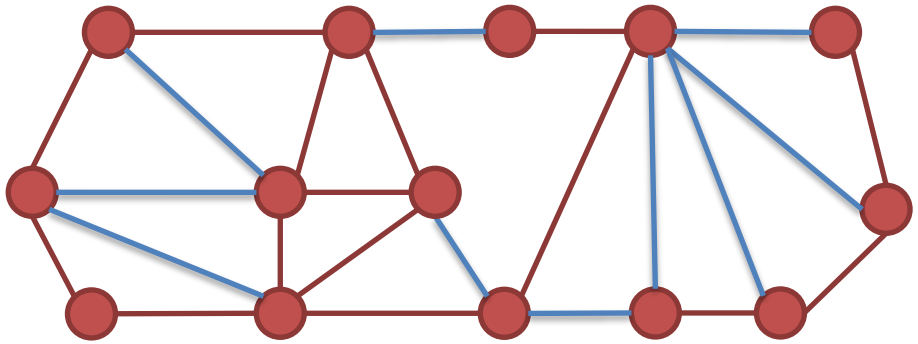
- A sub-graph $G' = (V', E')$ of another graph $G = (V, E)$ is induced by V' if:
  - for all $u, v \in V'$ if $(u, v) \in E$ then $(u, v) \in E'$.

- The red graph is a sub-graph induced by its vertex set.



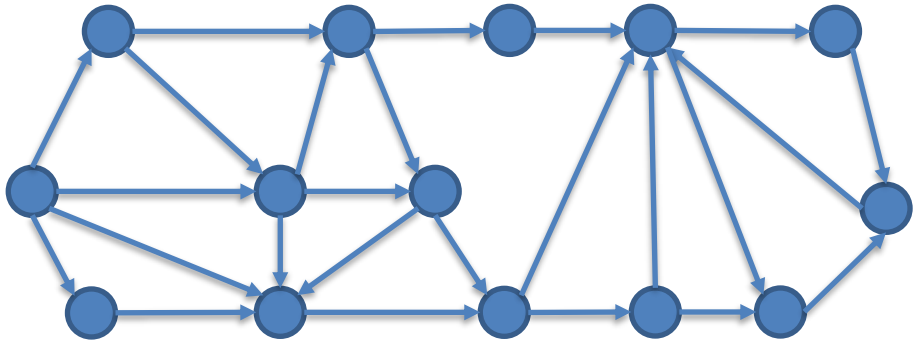
Graph Terminology

- A sub-graph $G' = (V', E')$ of another graph $G = (V, E)$ is spanning if $V' = V$.
- The red graph is a sub-graph spanning the original graph.



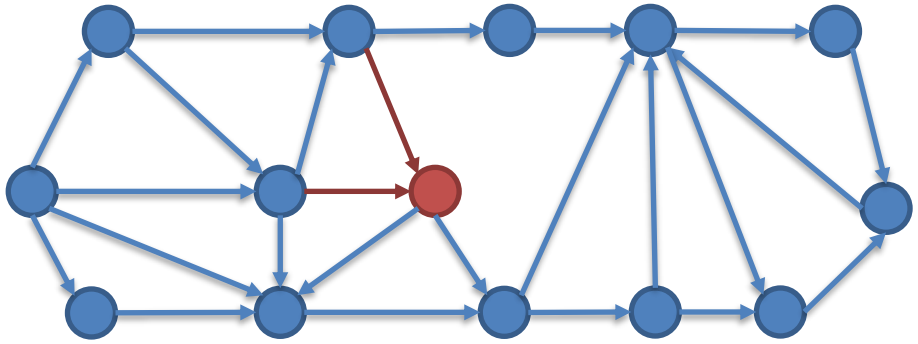
Digraph Terminology

- A directed graph is also known as a digraph.



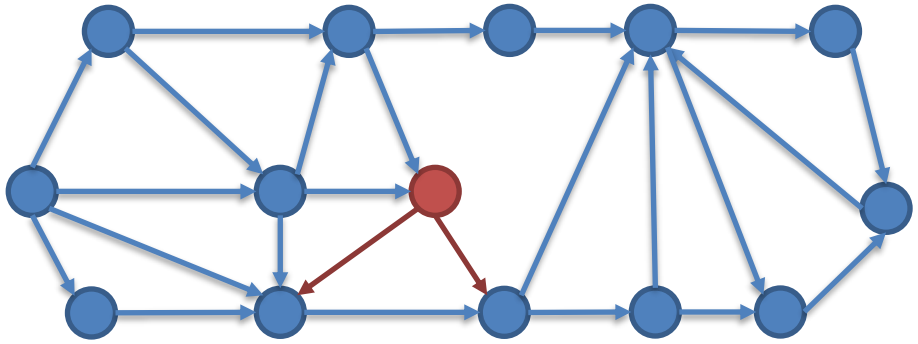
Digraph Terminology

- The indegree of a vertex (also known as fan in) is the number of incoming edges.
  - The indegree of this vertex is 2.



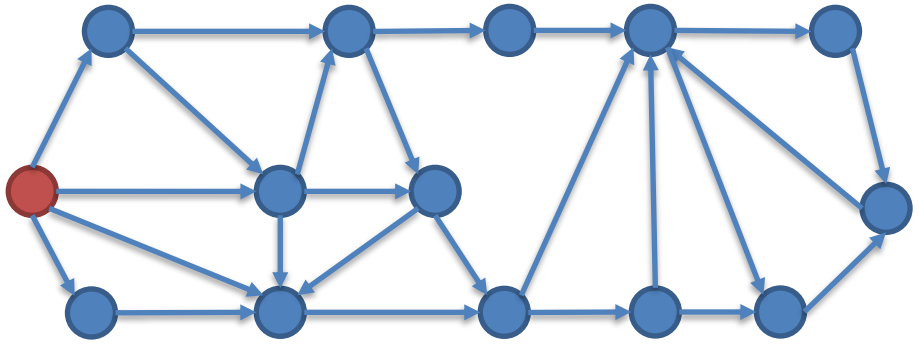
Digraph Terminology

- The outdegree of a vertex (also known as fan out) is the number of outgoing edges.
  - The outdegree of this vertex is 2.



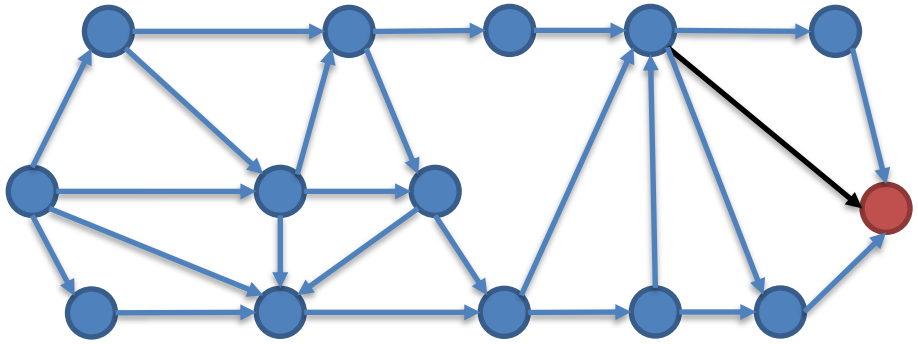
Digraph Terminology

- A vertex with an indegree of 0 is called a source.
  - This vertex is a source.



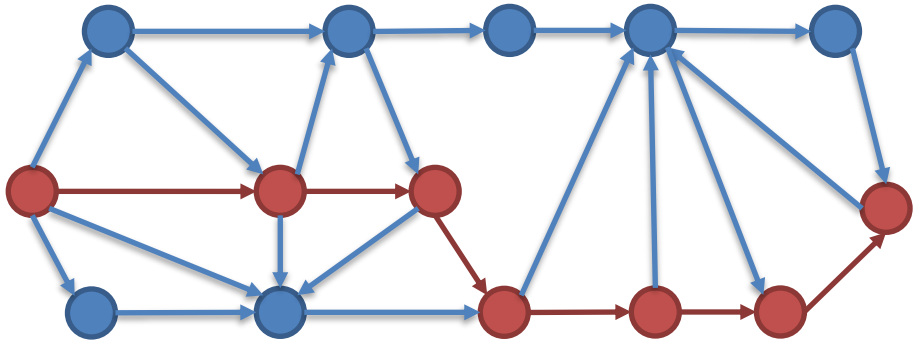
Digraph Terminology

- A vertex with an outdegree of 0 is called a sink.
  - This vertex is a sink after we swap the direction on the black edge.



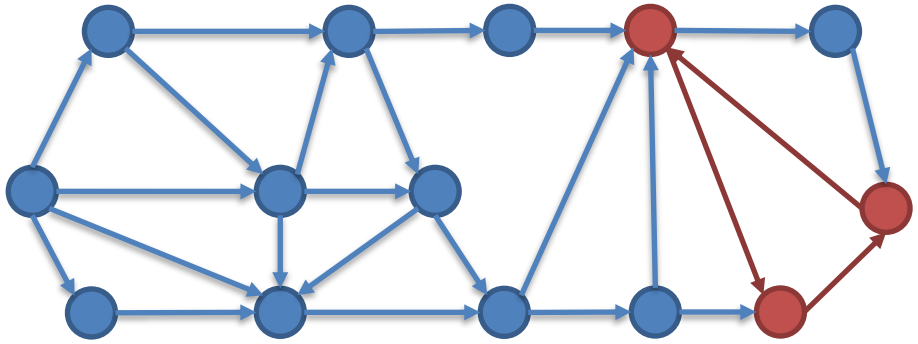
Digraph Terminology

- A directed path is a sequence of vertices $(v_1, v_2, \ldots, v_n)$ such that
$$\{(v_i, v_{i+1}) | 1 \leq i < n\} \subseteq E.$$
  - A simple directed path does not repeat vertices.



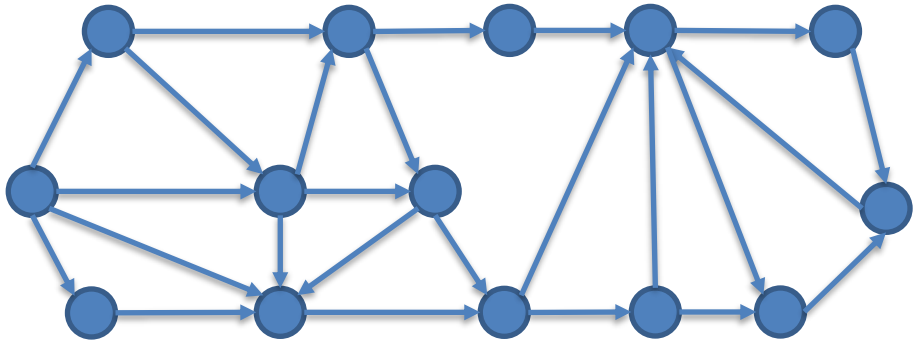
Digraph Terminology

- A directed cycle is a sequence of vertices $(v_1, v_2, \ldots, v_n)$ such that
$$\{(v_i, v_{i+1}) | 1 \leq i < n\} \subseteq E \ \wedge (v_n, v_1) \in E.$$
  - A simple directed cycle does not repeat vertices.



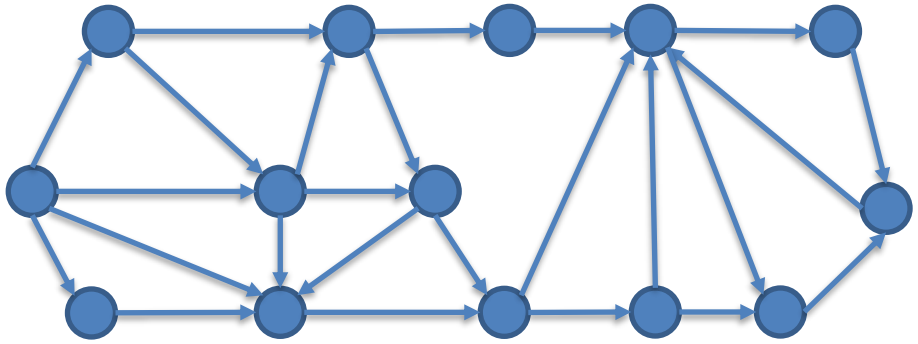
Digraph Terminology

- A directed graph is strongly connected if there exists a directed path from every vertex to every other vertex.
  - This graph is not strongly connected.



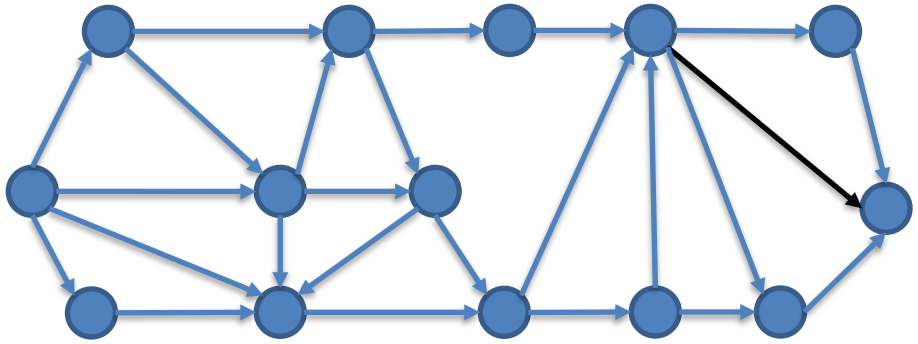
Digraph Terminology

- A directed graph is weakly connected if there exists a path from every vertex to every other vertex on the underlying undirected graph.
  - This graph is weakly connected.



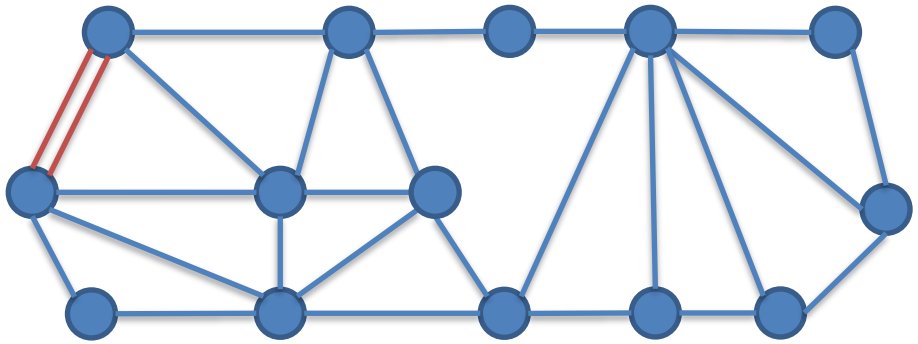
Kinds of Graphs

- A directed graph is a directed acyclic graph (DAG) if it contains no directed cycles.
  - This graph is a directed acyclic graph after we swap the direction on the black edge.



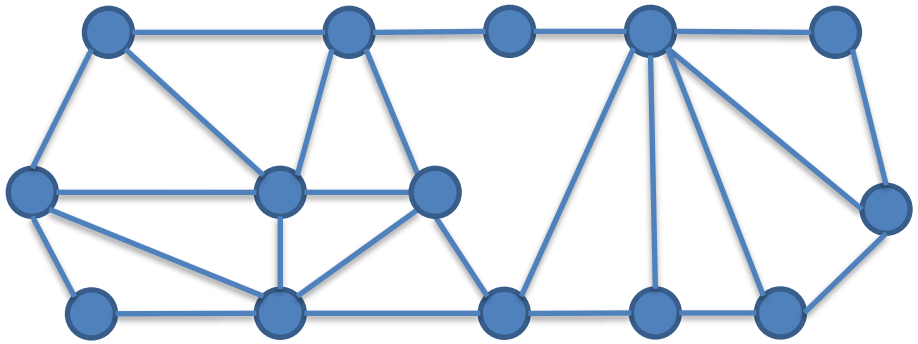
Kinds of Graph

- A multi-graph permits more than one edge between two vertices.
  - This is a multi-graph after we add another edge.



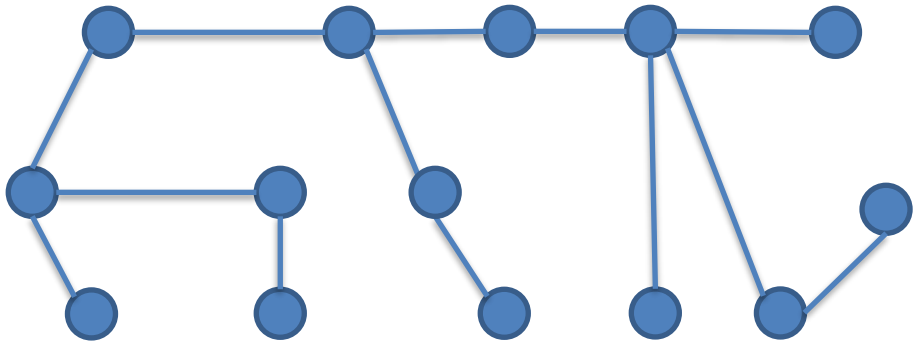
Kinds of Graph

- A graph is called planar if it can be drawn in two dimensions (i.e. on a plane) with no edges crossing over.
  - This graph is planar.



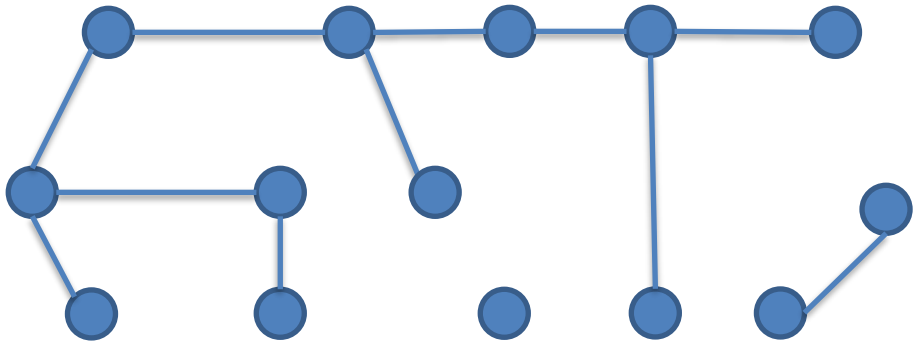
Kinds of Graph

- A tree is a connected undirected graph with no cycles.
  - This is a tree created from the original graph by deleting arbitrary edges that were in cycles.
  - This is also called a spanning tree because it includes all the original vertices.



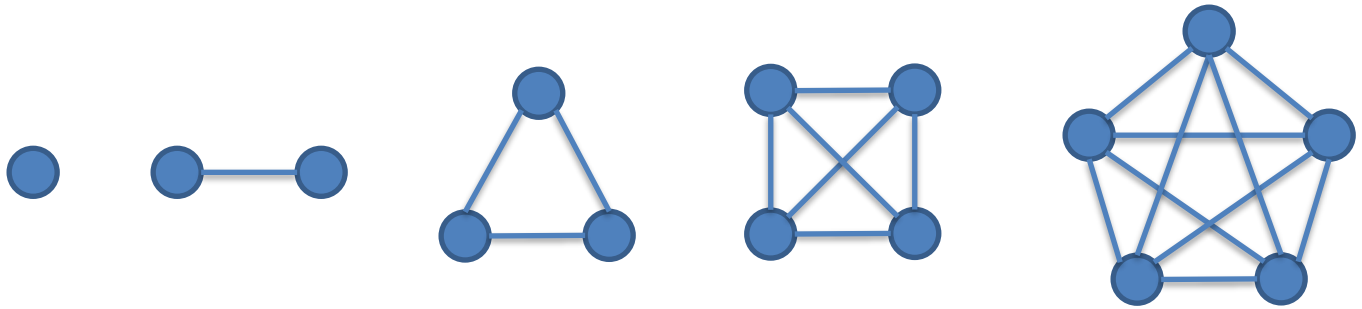
Kinds of Graph

- A forest is an undirected graph with no cycles.
  - This is a forest created from the original graph by deleting arbitrary edges that were in cycles.
  - This is also called a spanning forest because it includes all the original vertices.



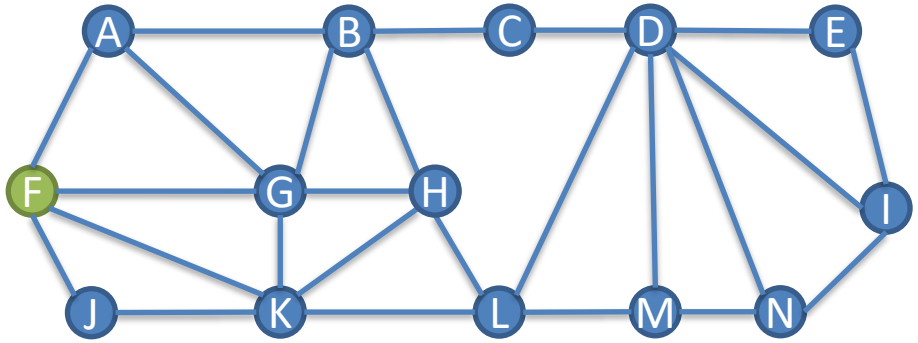
Kinds of Graph

- A complete graph is a graph that has every possible edge.
  - These are complete graphs of different sizes.



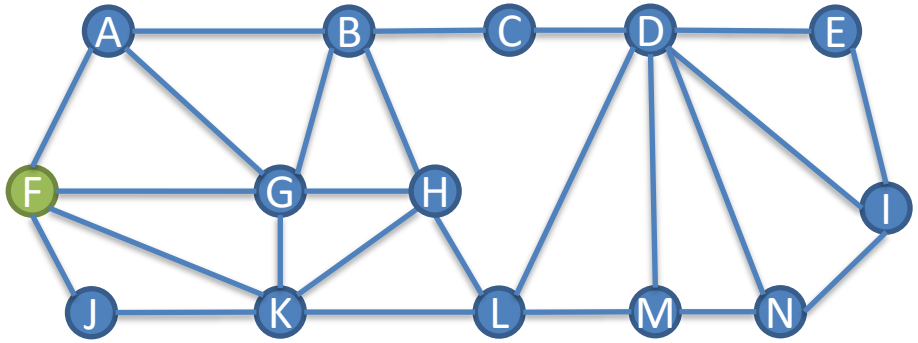
Depth First Traversal

- Depth first traversal:
  1. Start at an arbitrary vertex. Mark it as visited.
  2. Repeat this step until we can backtrack no further.
     1. If there is an unvisited vertex adjacent to the current vertex then traverse there and mark the vertex as visited.
     2. If there are no unvisited vertices, then backtrack to the last vertex visited before this one.
- This algorithm is recursively implemented in most cases. Otherwise a stack is used to keep track of previous vertices.



Breadth First Traversal

- Breadth first traversal:
  1. Start by adding an arbitrary vertex to a queue. Mark it visited.
  2. Repeat this step until the queue is empty.
     1. Remove the next vertex from the queue.
     2. Add all unvisited vertices that are adjacent to the current vertex to the queue. Mark each vertex added in this way as visited.



Graph Traversal

- Both depth first and breadth first traversals have a similar structure when implemented.

```
For each vertex v ∈ V:
      For each vertex u ∈ adj(v):
            Do some constant amount of work.
      End For
End For
```

- We can express the number of operations carried out by one of the traversals by this sum:

$$\sum_{v \in V} \sum_{u \in \text{adj}(v)} d$$

Graph Traversal

$$\sum_{v \in V} \sum_{u \in \text{adj}(v)} d = d \sum_{v \in V} |\text{adj}(v)|$$

- Notice that $|\text{adj}(v)| = \deg(v)$. So:

$$d \sum_{v \in V} |\text{adj}(v)| = d \sum_{v \in V} \deg(v)$$

- If we are adding up the degrees of each vertex, then each edge will be counted exactly twice (once for each endpoint). So (we call this the handshaking theorem):

$$\sum_{v \in V} \deg(v) = 2|E|$$

Graph Traversal

$$\sum_{v \in V} \sum_{u \in \text{adj}(v)} d = d \sum_{v \in V} |\text{adj}(v)| = d \sum_{v \in V} \deg(v) = d2|E|$$

- So in conclusion both depth first and breadth first traversals take $O(|E|)$ operations.
  - This means the number of operations can vary between sparse and dense graphs.

- Many algorithms have this same structure in whole or in part.
  - Prim's algorithm for finding the minimum spanning tree.
  - Dijkstra's algorithm for finding the shortest path between two points in a weighted graph.

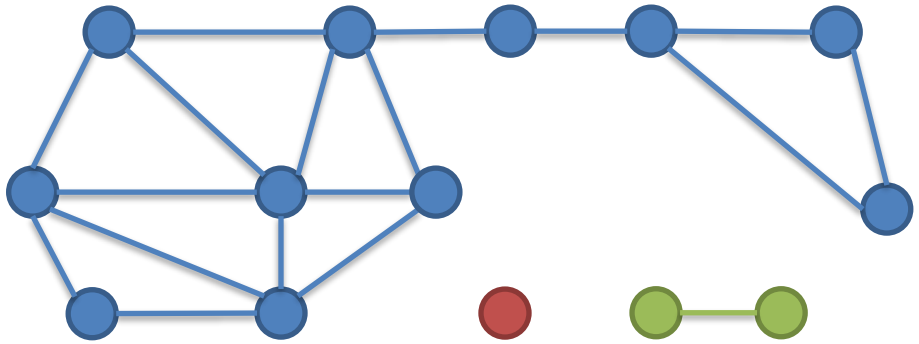
TCSS 342 – Data Structures

Graph Problems

Connected Graphs

- Given an undirected graph how can we check to see if it is connected?
  - For many problems we'll perform this check on our input graph first.

```
Connected
Input:  G = (V, E) an undirected graph
Output: "yes" if the graph is connected, and
        "no" otherwise
```



- This problem is stated as a decision problem.
  - A decision problem has a yes/no output.

- We could state a related non-decision problem that returns the connected components as follows:

```
Connected
Input:  G = (V, E) an undirected graph
Output: {G_i = (V_i, E_i)} a partitioning of G into connected sub-graphs
```

- These problems can be solved by applying a depth-first search.
  - Recall a depth-first search will take $O(|E|)$ operations.

Cycles

- Given a connected undirected graph how can we check to see if it has a cycle?

```
Cycle
Input:  G = (V, E) a connected undirected graph
Output: "yes" if the graph has a cycle, and
        "no" otherwise
```

```
Cycle
Input:  G = (V, E) a connected undirected graph
Output: C = (c_1, c_2, ..., c_n) a cycle of vertices, or
        "no" if no cycles exist
```

- Detecting a cycle can also be straightforwardly solved using a depth-first search.
  - Many algorithms use cycle detection and cycle recovery as a sub-problem.
  - Depending on the algorithm, and how clever we are, there may be a more efficient way to detect a cycle than a somewhat costly depth-first search.

Trees and Forests

- Recall that forests (trees) can be defined as (connected) graphs with no cycles.
  - These problems are the complement of the cycle detection problems.
  - As such we can use depth-first search to solve these problems as well.

```
Forest
Input:  G = (V, E) an undirected graph
Output: "yes" if the graph is a forest, and
        "no" otherwise
```

```
Tree
Input:  G = (V, E) a connected undirected graph
Output: "yes" if the graph is a tree, and
        "no" otherwise
```

- A related problem is finding a spanning tree of a graph.

```
Minimum Spanning Tree
Input:  G = (V, E, w) a weighted connected undirected graph
Output: m a minimum weight and T = (V, E') a spanning tree of G with weight m
```

Shortest Path

- A common problem in graphs is to find the shortest path between two vertices.

  Shortest Path
  Input: $G = (V, E)$ a connected undirected graph and $s \in V$ a source vertex and $d \in V$ a destination vertex
  Output: m the number of edges in the shortest path and $P = (p_1, \ldots, p_m)$ a path of vertices from s to d

- This version of the shortest path problem can be solved using a breadth-first search.
  - Recall breadth-first search also takes $O(|E|)$ operations.

  Shortest Path
  Input: $G = (V, E, w)$ a weighted connected undirected graph and $s \in V$ a source vertex and $d \in V$ a destination vertex and where $w(e) > 0$ for all $e \in E$
  Output: m a minimum weight and $P = (p_1, \ldots, p_k)$ a path of vertices from s to d where $m = \sum_{i=1}^{k-1} w(p_i, p_{i+1})$

- This problem is also called the "single-source single-destination shortest paths" problem.

Konigsberg Bridge Problem

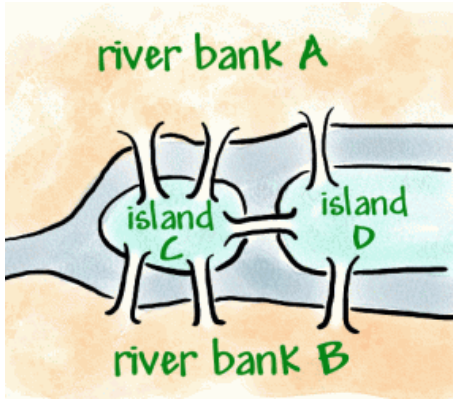
- In the Prussian city of Konigsberg (now a Russian city called Kaliningrad) there are seven famous bridges over the Pregl River. In 1735 the city wanted to promote tourism by creating a tour of the city that crossed each bridge exactly once and returned to the origin. After trying to construct such a walk on their own and failing the city asked the famous mathematician Leonard Euler to solve the problem.
  - His negative solution helped create graph theory.

Euler Cycle
Input: $G = (V, E)$ a connected undirected graph
Output: $E' = (e_1, \ldots, e_k)$ a permutation of the edge set constituting a cycle, or "no" if no such permutation exists

Euler Path
Input: $G = (V, E)$ a connected undirected graph
Output: $E' = (e_1, \ldots, e_k)$ a permutation of the edge set constituting a path, or "no" if no such permutation exists

- Euler was able to prove:
  - A graph has an Euler cycle if and only if every vertex has even degree.
  - A graph has an Euler path if and only if there exist exactly two odd degree vertices.

- In addition he provided algorithms for finding Euler cycles and paths in graphs that contain them.



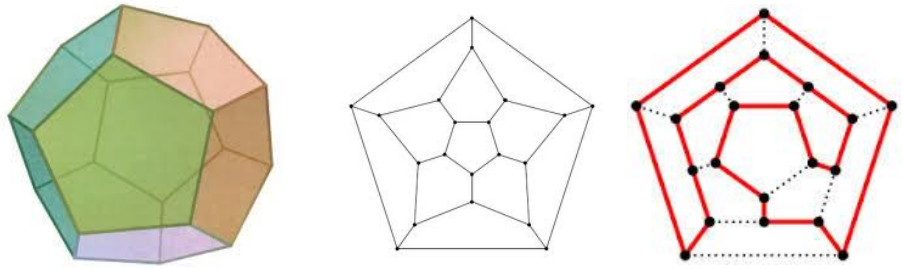
Hamiltonian Cycle (Path)

- William Hamilton was an Irish mathematician who proposed a game he called the Icosian game.
  - In this game you had a dodecahedron with a pin on each intersection.
  - The goal was to connect a string that visits every pin exactly once and returns to the first pin.
  - This type of cycle is called a Hamiltonian cycle.

Hamiltonian Cycle
Input: $G = (V, E)$ a connected undirected graph
Output: $V' = (v'_1, \ldots, v'_n)$ a permutation of the vertex set constituting a cycle, or
"no" if no such permutation exists

Hamiltonian Path
Input: $G = (V, E)$ a connected undirected graph
Output: $V' = (v'_1, \ldots, v'_n)$ a permutation of the vertex set constituting a path, or
"no" if no such permutation exists

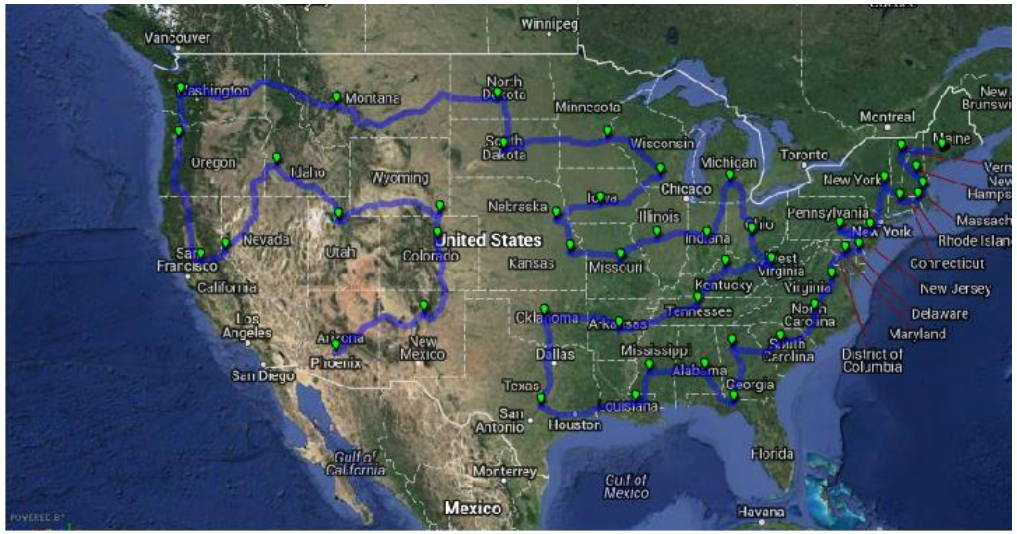
- Unlike Euler's problem this problem is notoriously difficult to solve.
  - The best-known solutions to this problem require exponential operations.
  - This is an example of an NP-complete problem.



Travelling Salespeople

- The travelling salesperson problem is a well-known variant of the Hamiltonian cycle problem.

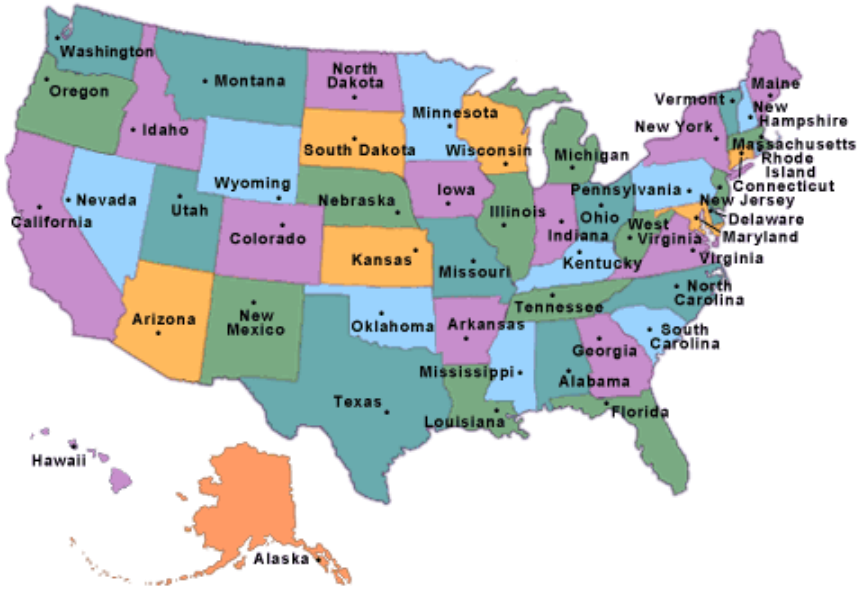
Travelling Salesperson
Input: $G = (V, E, w)$ a weighted connected undirected graph
Output: m a minimum cost and $V' = (v'_1, \ldots, v'_n)$ a permutation of the vertex set constituting a path (cycle) with total cost m, or "no" if no such permutation exists



Graph Coloring

Graph Coloring
Input: $G = (V, E)$ an undirected graph
Output: m a minimum number of colors and $c(v)$ a color function that maps vertices to colors from the set $\{c_1, \ldots, c_m\}$ such that if $(u, v) \in E$ then $c(u) \neq c(v)$





Sub-Graph Problems

Clique
Input: $G = (V, E)$ an undirected graph and an integer $k > 1$
Output: "yes" if there exists a complete sub-graph of G of size k
"no" otherwise

Independent Set
Input: $G = (V, E)$ an undirected graph and an integer $k > 1$
Output: "yes" if there exists a sub-graph of G of size k with no edges
"no" otherwise

Sub-Graph Isomorphism
Input: $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ two connected undirected graphs
Output: "yes" if there exists a sub-graph of G_2 that is isomorphic to G_1
"no" otherwise